DeltaX2 Control Application

User & Administrator Manual

Software version	0.1.0
Target platform	Raspberry Pi 5, 7 ” touch screen (800×480)
Robot	Delta X 2
Date	July 2026

This manual covers installation, configuration and daily operation of the DeltaX2 seeding-robot control application.

Contents

1 Introduction

The DeltaX2 control application drives a **Delta X 2** delta-arm robot used for automated seeding of paper-pot trays. It is written in Rust with the Slint UI toolkit and is designed for a dedicated operator terminal: a Raspberry Pi 5 with a 7 ” touch screen running in kiosk mode.

The application provides:

- a guided, touch-friendly **seeding workflow** (select tray, load seeds, run, pause/resume, stop);
- a **calibration screen** for manual jogging of the X, Y and Z axes with selectable step sizes and mechanical homing;
- **software safety limits**: every movement is checked against configurable boundaries before any command is sent to the robot;
- live **position readout** and a permanent **status bar** with a connection indicator;
- plain-text configuration through a single `config.toml` file — new tray layouts can be added without recompiling.

1.1 Intended audience and structure

Chapters 3 to 5 are aimed at the *administrator* who installs and configures the system (keyboard and SSH access assumed): Chapter 3 covers building the software, Chapter 4 its deployment on the Raspberry Pi terminal, and Chapter 5 the `config.toml` settings. Chapter 6 is aimed at the *daily operator*, who interacts with the machine exclusively through the touch screen. Chapter 7 helps diagnose common problems. Appendix A contains the complete G-code and communication protocol specification of the Delta X 2, and Appendix B lists current development limitations.

1.2 Typographic conventions

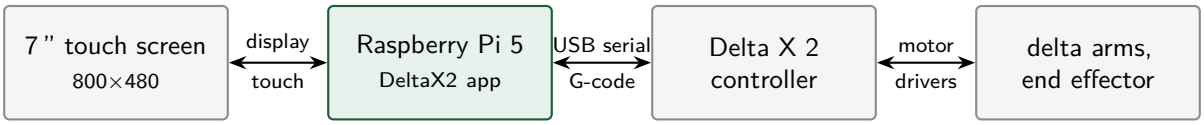
- `monospace` — file names, commands, configuration keys, on-wire commands.
- **Bold** — names of buttons and screens as they appear on the display.
- Shell commands prefixed with `$` are typed in a terminal (locally or over SSH).

Warning
The Delta X 2 is a moving machine. Keep hands and tools out of the working area while the robot is powered. The software enforces motion limits, but software protections are no substitute for physical caution. An additional hardware emergency-stop button that cuts motor power is strongly recommended (see Appendix B).

2 System overview

2.1 Hardware components

The system consists of four hardware elements connected in a chain: the operator's touch screen, the Raspberry Pi running this application, the Delta X 2 controller board, and the mechanics it drives.

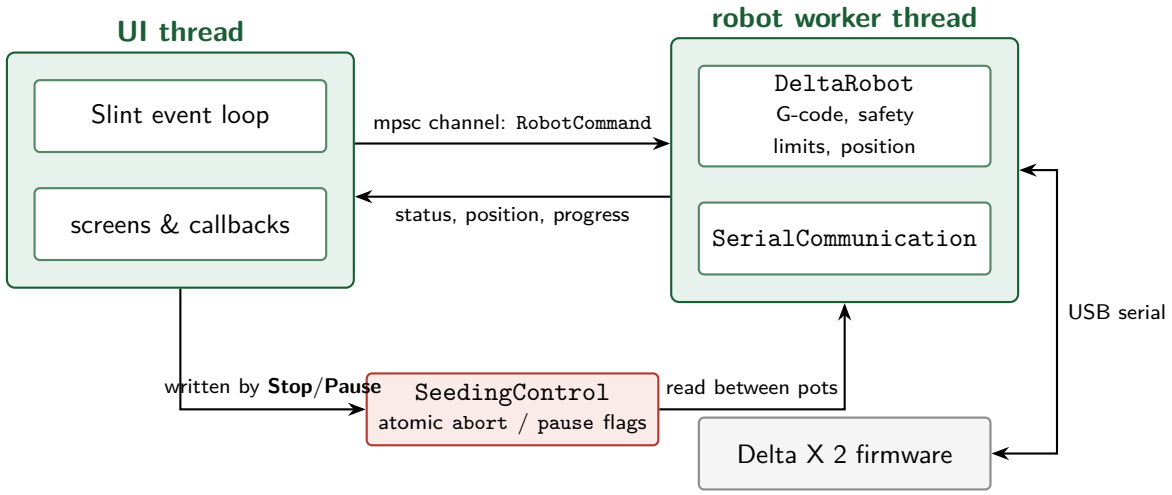


Component	Role
Delta X 2 robot	Delta-arm manipulator performing the seeding motion. Controlled over a USB serial link (G-code, 115 200 baud, Appendix A).
Raspberry Pi 5	Runs the DeltaX2 application (Chapter 4).
7 " touch screen	Operator interface, 800 × 480 pixels, touch only.
Seeding plates	Interchangeable paper-pot trays; their geometry is described in `config.toml` (Section 5.4).

2.2 Software architecture

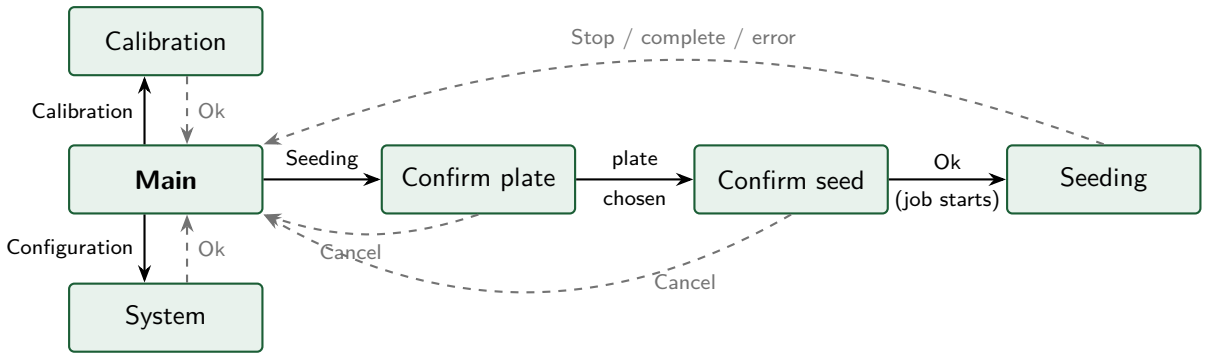
The application consists of two threads:

- The **UI thread** runs the Slint event loop and reacts to touch input. It never talks to the serial port directly, so the interface stays responsive at all times.
- The **robot worker thread** owns the serial connection. It executes one command at a time (connect, jog, home, seed a plate) and reports status, position and progress back to the UI.



Commands travel from the UI to the worker through a queue and are executed strictly one at a time. The pause/abort flags take a separate path on purpose: during a seeding job, the worker is busy inside the job loop and would not look at its queue, so the **Stop** and **Pause** buttons write shared flags that the loop checks between pots. The robot therefore finishes the pot it is currently working on and then stops or waits.

2.3 Screen map



Screen	Purpose
Main	Home screen; entry to the three functions below.
Seeding flow	Three steps: select the plate type, confirm seeds are loaded, then watch/control the running job.
Calibration	Manual jogging, step-size selection, homing, position readout.
System	System utilities (serial-port diagnostics).

A status bar is visible at the bottom of every screen. It shows a • green / • red connection indicator and the most recent status or error message.

3 Installation

This chapter covers building the application from source and running it on a development machine. Deployment on the Raspberry Pi operator terminal is the subject of Chapter 4.

3.1 Prerequisites

- **Target hardware** — a Raspberry Pi 3, 4 or 5 running a *64-bit* operating system (the build target is `aarch64-unknown-linux-gnu`), fitted with the official Raspberry Pi Touch Display (7 inch, 800 × 480). The Pi 3 is the performance floor. Release builds are compiled natively on the Pi; there is no cross-compilation setup.
- **Rust toolchain** — install via `https://rustup.rs`; distribution packages are usually too old. The project uses Rust edition 2024 and declares its minimum supported Rust version in `Cargo.toml` (`rust-version`, currently 1.90, driven by the dependency tree); `cargo` refuses to build with an older toolchain.
- **Build dependencies** — a C compiler and the graphics/input development libraries required by Slint. On Debian-based systems:

```
$ sudo apt install build-essential cmake libfontconfig1-dev \
      libfreetype6-dev libxkbcommon-dev libinput-dev \
      libudev-dev libgbm-dev
```

- **Serial port access** — the user running the application must be able to open the robot's serial device:

```
$ sudo usermod -aG dialout $USER    # log out and back in afterwards
```

3.2 Building from source

```
$ git clone <repository-url> deltax2
$ cd deltax2
$ cargo build --release
```

The binary is produced at `target/release/deltax2`.

For anyone modifying the source: the project has no CI, so the following quality gate must pass locally before a change is considered done:

```
$ cargo build && cargo clippy --all-targets -- -D warnings \
      && cargo fmt --check && cargo test
```

> **Note**
> The application reads `config.toml` from its *current working directory* at start-up, not from the directory containing the binary. Always start it from a directory that contains a valid `config.toml` (this matters for the systemd unit in Section 4.6, which sets `WorkingDirectory` explicitly).

3.3 Desktop test run

For development on a normal Linux desktop, connect the robot (or leave it disconnected — the UI starts regardless and shows *Disconnected*), then:

```
$ cargo run
```

Configure the serial port and tray layouts as described in Chapter 5 before first use with real hardware.

4 Raspberry Pi deployment

This chapter describes how to deploy and run the DeltaX2 application on a Raspberry Pi with a 7 ” touch screen (800×480 resolution) as a dedicated, touch-only operator terminal.

4.1 Operating system

Use **Raspberry Pi OS (64-bit)**. The Lite version is suitable — and preferred — if you run the application in kiosk mode without a full desktop environment: the application can render directly to the screen (Section 4.4).

4.2 System dependencies

When building the application on the Pi, or running it with the default renderer, install the following system libraries first:

```
$ sudo apt-get update
$ sudo apt-get install -y build-essential cmake \
      libfreetype6-dev libfontconfig1-dev libssh-dev \
      libxkbcommon-dev libinput-dev libudev-dev libgbm-dev
```

4.3 Building on the Pi and cross-compilation

The straightforward approach is to build natively on the Pi 5 (a release build takes a few minutes): copy or clone the repository onto the Pi and run `cargo build -release`.

Building on the Pi can be slow on older models; you can instead cross-compile from a faster machine using `cross-rs`:

```
$ cargo install cross
$ cross build --target aarch64-unknown-linux-gnu --release
```

Then copy the resulting binary *and* `config.toml` to the Pi.

4.4 Performance and rendering

Slint supports multiple rendering backends. On a Raspberry Pi, the **`linuxkms`** backend is often the most performant, as it renders directly to the screen without X11 or Wayland overhead.

To run with LinuxKMS:

1. Ensure your user is in the `video` and `input` groups:

```
$ sudo usermod -aG video,input,render $USER
```

2. Run the application with the backend selected via the environment:

```
$ SLINT_BACKEND=linuxkms ./deltax2
```

Note

The `linuxkms` backend is an optional Slint feature and must be enabled at compile time. If `SLINT_BACKEND=linuxkms` reports an unknown backend, add the feature to `Cargo.toml` and rebuild:

```
slint = { version = "1.14", features = ["backend-linuxkms"] }
```

When running under a desktop session (X11 or Wayland) instead, no environment variable is needed; the default `winit` backend is used.

4.5 Kiosk mode

For daily operation the application should fill the screen without window decorations. This is controlled by the configuration file (Chapter 5):

```
[ui]
kiosk_mode = true
```

4.6 Automatic start with systemd

To run the application as a standalone interface on boot, create the unit file `deltax2.service` in `/etc/systemd/system/` (adjust user and paths):

```
[Unit]
Description=DeltaX2 seeding robot control UI
After=multi-user.target

[Service]
User=pi
WorkingDirectory=/home/pi/deltax2
Environment=SLINT_BACKEND=linuxkms
ExecStart=/home/pi/deltax2/target/release/deltax2
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

The application looks for `config.toml` next to the executable and in the working directory (and accepts an explicit `--config <path>`), so the `WorkingDirectory` line above is a convenience rather than a requirement: a copy of `config.toml` in `/home/pi/deltax2` (beside the binary) is found either way, and a manual SSH launch from any directory still starts.

Then enable and start it:

```
$ sudo systemctl daemon-reload
$ sudo systemctl enable --now deltax2.service
$ journalctl -u deltax2 -f        # follow the application log
```

All console messages of the application (connection results, executed serial commands, port listings) appear in the systemd journal, which is the primary diagnostic channel when working over SSH.

If you use a display manager and a desktop session instead of the bare-metal `linuxkms` setup, an `.xsession` file or the desktop environment's autostart mechanism can launch the application in the same way (without the `SLINT_BACKEND` variable).

4.7 Touch screen calibration

If the touch input is inverted or misaligned, configure `libinput` with a udev rule that sets the `LIBINPUT_CALIBRATION_MATRIX`, e.g. in `/etc/udev/rules.d/99-touch.rules`:

```
ENV{ID_INPUT_TOUCHSCREEN}=="1", \
  ENV{LIBINPUT_CALIBRATION_MATRIX}="1 0 0 0 1 0"
```

The six values form the first two rows of a 3×3 transformation matrix; for example `-1 0 1 0 -1 1` rotates the input by 180°.

4.8 Serial port permissions

If the application cannot open the serial port, ensure the user has permission to access serial devices:

```
$ sudo usermod -aG dialout $USER
```

Log out and back in (or restart the systemd service) for the change to take effect.

5 Configuration

All settings live in a single file, `config.toml`, read once at start-up from the application's working directory. **Changes take effect after restarting the application** (`sudo systemctl restart deltax2` on a kiosk installation).

If the file is missing or contains a syntax error, the application prints an error and exits; on a kiosk installation check `journalctl -u deltax2` for the reason.

5.1 Serial connection — `[serial]`

```
[serial]
port = "/dev/ttyUSB0"    # robot serial device
baud_rate = 115200       # must match the robot firmware
```

Key	Type	Meaning
`port`	string	Serial device path. Typically `/dev/ttyUSB0` or `/dev/ttyACM0` on Linux. Use the **List com** button (Section 6.5) or `ls /dev/tty*` to find it.
`baud_rate`	integer	Communication speed; the Delta X 2 firmware uses 115 200 (Appendix A).

> **Tip**
> USB device names can change between reboots when several adapters are connected. For a robust kiosk setup, use a stable symlink from `/dev/serial/by-id/...` as the `port` value.

5.2 User interface — `[ui]`

```
[ui]
kiosk_mode = false    # true: borderless full-screen for the Pi touch screen
```

5.3 Safety limits — `[robot]`

```
[robot]
limit_min = { x = -160.0, y = -160.0, z = -200.0 }
limit_max = { x =  160.0, y =  160.0, z =    0.0 }
```

Every jog and seeding movement is checked against these boundaries *before* any command is sent to the hardware; a move that would leave the box is refused with the status message *Movement out of safety limits*. The values must lie within the physical workspace of the robot, described in Section A.6.

Warning
The coordinate origin is established by homing (`G28`): after homing, the head is at the *top centre* of the working volume, at $X{=}0$, $Y{=}0$, $Z{=}0$. The Z axis points *upwards*: all working positions below the home position have **negative Z**. Consequently `limit_max.z` should normally stay at `0.0` and `limit_min.z` defines the lowest permitted point. Set the limits to match your actual machine and tooling before first use.

5.4 Seeding plates — `[[plates]]`

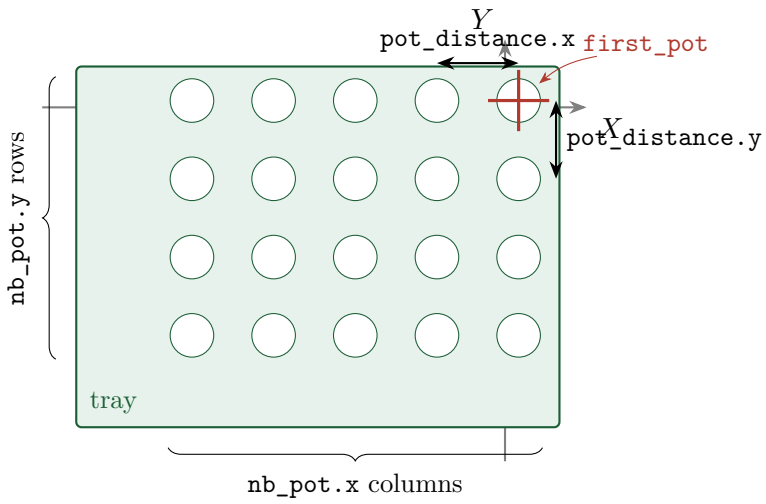
Each `[[plates]]` block describes one tray layout and appears as a selectable button in the seeding workflow. Add as many blocks as you have tray types.

```
[[plates]]
name = "77pots"                          # button label in the UI
plate_size   = { x = 500, y = 700, z = 40 }  # tray dimensions in mm
first_pot    = { x = 7, y = 24 }         # centre of the first pot (robot coords, mm)
pot_distance = { x = 10, y = 12 }        # centre-to-centre spacing in mm
nb_pot       = { x = 7, y = 11 }         # grid size: pots in X and Y
```

The application computes the position of pot (i, j), with $0 \le i <$ `nb_pot.x` and $0 \le j <$ `nb_pot.y`, as:

$$x_{ij} = \texttt{first\_pot.x} - i \cdot \texttt{pot\_distance.x}, \qquad y_{ij} = \texttt{first\_pot.y} - j \cdot \texttt{pot\_distance.y}$$

Note the *subtraction*: the grid extends from the first pot towards **negative** X and Y. The first pot is therefore the one with the largest coordinates, and the whole grid must fit inside the safety limits of Section 5.3.



5.4.1 Worked example: adding a new tray type

Suppose a new tray has 6×9 pots, 15 mm apart in X, 14 mm apart in Y, and its first pot centre sits at robot coordinates $(30, 50)$:

```
[[plates]]
name = "54pots"
plate_size   = { x = 500, y = 700, z = 40 }
first_pot    = { x = 30, y = 50 }
pot_distance = { x = 15, y = 14 }
nb_pot       = { x = 6, y = 9 }
```

Check the extremes: the last pot lies at $x = 30 - 5 \cdot 15 = -45$ and $y = 50 - 8 \cdot 14 = -62$ — inside the default ± 160 mm limits. Restart the application; the new **54pots** button appears in the plate selection screen.

6 Daily operation

This chapter describes operation through the touch screen only; no keyboard is required. The navigation between screens is summarized in the screen map of Section 2.3.

6.1 Start-up and the status bar

After power-on (with the systemd unit of Section 4.6), the application starts automatically and immediately shows the **Main** screen while it connects to the robot in the background. The status bar at the bottom shows:

- a **connection indicator**: green *Connected* once the robot has answered the identification handshake, red *Disconnected* otherwise;
- the latest **status message**, e.g. *Connecting…*, *Connected*, *Move OK*, *Homing complete*, or an error text.

> **Note**
> If the robot was off when the terminal started, the status shows *Connection … failed*. Power the robot on, then restart the application (power-cycle the terminal, or `sudo systemctl restart deltax2` over SSH). An in-UI reconnect button is planned (Appendix B).

6.2 Main screen

Three buttons:

Button	Function
Seeding	Starts the guided seeding workflow (Section 6.3).
Calibration	Opens manual jog and homing (Section 6.4).
Configuration	Opens system utilities (Section 6.5).

6.3 Seeding a plate

1. Touch **Seeding** on the main screen.
2. **Install the seeding plate**: place the tray in its mechanical reference position, then touch the button matching its type (e.g. **77pots**). The choice is recorded and shown in the status bar. **Cancel** returns to the main screen.
3. **Load the seeds**: fill the seed supply, then touch **Ok** to start the job — the robot homes first and then processes the tray pot by pot. **Cancel** returns to the main screen without moving the robot.

4. **Seeding in progress**: the screen shows the live progress (*Pot 12 / 77*) and three controls:
   - **Pause** (‖) — the robot finishes the current pot and waits. The status bar shows *Seeding paused*.
   - **Continue** (▷) — resumes a paused job.
   - **Stop** (□) — aborts the job: the robot finishes the current pot, then stops. The display returns to the main screen and the status bar shows *Stopping…* followed by *Seeding stopped*.
5. On completion the display returns to the main screen and the status bar shows *Seeding complete*.

Warning
Stop waits for the current pot to finish; it does not cut power. For immediate mechanical arrest in an emergency, use the machine's hardware emergency stop (if fitted) or remove robot power.

6.4 Calibration and manual movement

The **Calibration** screen provides:

- **Step size selector** — four buttons: 0.01, 0.1, 1 or 5 mm. The selected value applies to every subsequent jog.
- **XY pad** — arrow buttons jog the head in X and Y by one step per touch; the centre button homes all axes (`G28`).
- **Z column** — up/down jog the head vertically; the centre button also triggers homing.
- **Cart column** — up/down adjust the rotation axis; the centre button resets its displayed value (see the note below).
- **Position panel** — live display of the logical X, Y, Z and Cart coordinates in mm.

Each jog is checked against the safety limits; a refused move leaves the robot untouched and shows *Movement out of safety limits* in the status bar. After homing, the position display resets to 0.000 on all axes.

Note
In the current software version the **Cart** axis is tracked on screen but not yet driven on the hardware (Appendix B).

Tip
Jog commands are queued and executed in order. On a touch screen it is easy to tap faster than the robot moves — prefer single deliberate taps and watch the position readout; each completed move updates the display.

6.5 System screen

The **Configuration** button opens the system screen, which currently offers:

- **List com** — enumerates the serial ports detected on the system. The result is written to the application log (visible with `journalctl -u deltax2` over SSH), which helps an administrator find the correct `port` value for `config.toml`.

7 Troubleshooting

7.1 Common problems

Symptom	Cause and remedy
Status: *Connection to ... failed: Permission denied*	The user lacks serial-port rights. `sudo usermod -aG dialout <user>`, then log out/in (or restart the service).
Status: *Connection ... failed: No such file or directory*	Wrong `port` in `config.toml`, or the USB cable is unplugged. Use **List com** / `ls /dev/serial/by-id/` to identify the device.
Status: *Device on ... did not respond correctly to IsDelta*	A serial device was found but it is not (or not yet) the robot: wrong port, wrong baud rate, or robot still booting. Verify `baud_rate = 115200`, power-cycle the robot, restart the application.
Status: *Timeout waiting for 'ok' from robot*	The robot stopped answering mid-command: cable disturbed, firmware halted, or mechanical fault. Check the machine, then restart the application and re-home before continuing.
Status: *Movement out of safety limits*	The requested jog would leave the configured box. If the limits are too tight for your tooling, adjust `[robot]` in `config.toml` (Section 5.3).
Application does not start on the Pi (blank screen)	Check `journalctl -u deltax2`. Typical causes: missing/invalid `config.toml` in the `WorkingDirectory`, or the binary was built without the `linuxkms` backend feature while `SLINT_BACKEND=linuxkms` is set (Section 4.4).
Touch input misaligned or rotated	Configure `LIBINPUT_CALIBRATION_MATRIX` via a udev rule (Section 4.7).

7.2 Diagnostics over SSH

```
$ journalctl -u deltax2 -f          # live application log
$ ls -l /dev/serial/by-id/          # stable names of USB serial adapters
$ sudo systemctl restart deltax2    # restart the UI after config changes
```

The log contains every serial command sent (`Serial write: ...`), all status messages shown in the UI, and the output of **List com**.

A Delta X 2 G-code specification

This appendix specifies the G-code commands and the communication protocol of the Delta X 2 robot, and describes how the DeltaX2 application uses them.

A.1 Communication protocol

Parameter	Value
Baud rate	115 200
Line ending	`\n` (newline)
Handshaking	The robot typically responds to commands. For critical synchronization, use the `FEEDBACK` parameter (Section A.4).

A.2 Identification and status

A.2.1 `IsDelta`

Queries the device to confirm it is a Delta X robot.

- **Response**: `YesDelta`

A.2.2 `DeltaState`

Queries the current operating state of the robot. Possible responses:

Response	Meaning
`Free`	Idle and ready.
`Running`	Executing a command.
`Wait`	Waiting for input or synchronization.
`Almostdone`	Near the end of a movement.
`Done`	Movement or command completed.

A.2.3 `Position`

Returns the current coordinates of the robot.

- **Response format**: `X:<val> Y:<val> Z:<val> W:<val> U:<val>`

A.3 Movement commands (G-codes)

A.3.1 G00 / G01: linear movement

Moves the end-effector to the specified coordinates.

- **Syntax**: `G01 X<val> Y<val> Z<val> F<speed>`
- **Example**: `G01 X10.5 Y-20.0 Z-150.0 F2000`

A.3.2 G28: homing

Moves all axes to their home position (top endstops).

- **Origin** ($X0\,Y0\,Z0$): after homing, the robot is positioned at the top centre of the workspace.
- **Z axis**: moves downward with negative values. $Z{=}0$ is the top; $Z{=}{-}200$ is near the bottom of the workspace.

A.3.3 G90: absolute positioning

Interpret subsequent coordinates as absolute positions relative to the origin.

A.3.4 G91: relative positioning

Interpret subsequent coordinates as displacements from the current position (useful for jogging).

A.4 Synchronization (FEEDBACK)

`FEEDBACK:<string>` can be appended to any G-code command. The robot echoes `<string>` back to the serial port once the command has been successfully executed.

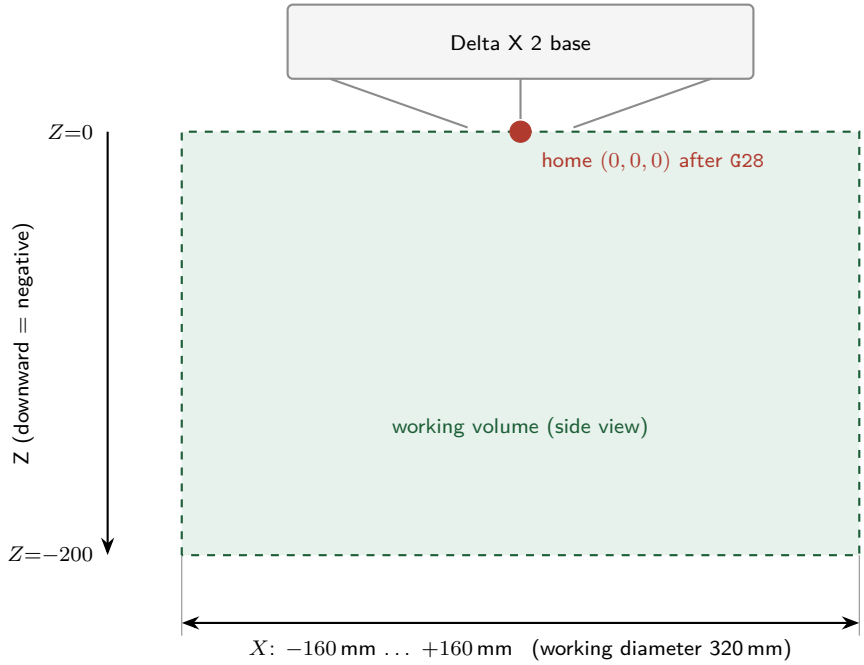
- **Example**: `G01 X0 Y0 Z-50 FEEDBACK:done`
- **Response**: `done` (sent after the move finishes).

A.5 End-effector controls (M-codes)

Command	Meaning
M03	Output on — activates the end effector (e.g. opens gripper, starts vacuum).
M05	Output off — deactivates the end effector.

A.6 Physical workspace (SP-X2)

Axis	Range
X	−160 mm to +160 mm
Y	−160 mm to +160 mm
Z	−200 mm to 0 mm (0 at the top)
Working diameter	320 mm



A.7 How the application uses the protocol

At connection time the application sends `IsDelta` and requires the `YesDelta` answer within two seconds. It then issues `Position` to read the head coordinates back from the firmware and adopt them as its tracked position, so a robot that was moved by hand while disconnected is never driven against a stale estimate; if no position is returned, the application keeps the state "unknown" and requires a homing before the first jog. Every command it issues afterwards carries a `FEEDBACK:ok` suffix, and the next command is only sent after the robot has echoed `ok`. Jog moves from the calibration screen are wrapped in `G91` … `G90` so they execute as relative displacements; homing uses `G28`. The `M03`/`M05` end-effector commands are reserved for the per-pot seeding action (Appendix B).

B Development status and known limitations

This software is under active development (version 0.1.0). The following limitations are known and planned to be addressed:

- **Per-pot seeding action**: the movement/tool sequence executed at each pot position is not yet implemented; a seeding run currently homes the robot and steps through the pot grid without dispensing. The implementation will use the `M03`/`M05` end-effector commands (Appendix A).
- **Cart (rotation) axis**: tracked in the interface but not yet driven on the hardware.
- **Reconnection**: if the robot is unavailable at start-up, the application must be restarted to retry; an in-UI reconnect is planned.
- **Port listing**: **List com** reports to the application log only, not on screen.
- **Hardware emergency stop**: a physical mushroom-head stop button that cuts robot motor power (with a second contact informing the application via GPIO) is recommended and planned, but not yet part of the design.